FOURIER-PRICING UNDER DISCRETE FOURIER TRANSFORM (DFT)

Reading Time	45 minutes
Prior Knowledge	Fourier Transform, Characteristic Function (CF), Derivative Pricing
Keywords	Discrete Fourier Transform (DFT), Fast Fourier Transform (FFT), Carr-Madan (1999)

Now we know about the power of Fourier methods for getting analytical solutions for vanilla option prices. As we a mentioned, there are several ways to leverage Fourier tools, including continuous- and discrete-time transforms. In this lesson we will cover the practical implementation of the discrete-time version, which is the basis of the Carr-Madan (1999) pricing method.

1. Discrete Fourier Transform (DFT)

Before jumping straight to the coding part, let's learn a little bit more about how the Discrete Fourier Transform (DFT) works.

You may remember Carr and Madan's (1999) expression for the price of a Call option from Lesson 1
$$ C_0 = \frac{e^{-\alpha k}}{\pi} \int_{0}^{\infty} e^{-i\nu k} \Psi(\nu) \,d\nu\ $$

To solve this integral, however, we need to use some kind of **discretization technique**. For example, the **trapezoidal rule**:

$$ \int_a^b f(x) \,dx \approx T(f,\eta) = \frac{\eta}{2}(f(x_0) + 2f(x_1) + 2f(x_2) + ... + 2f(x_{N-2}) + 2f(x_{N-1}) + f(x_N)) $$

Truncating the upper limit of the integral above to B, and applying the trapezoidal rule, yields the DFT:

$$ C_T(k) = \frac{e^{-\alpha k}}{\pi} \int_{0}^{\infty} e^{-i\nu k} \Psi(\nu) \,d\nu\

$$ \approx \frac{e^{-\alpha k}}{\pi} \int_{0}^{B} e^{-i\nu k} \Psi(\nu) \,d\nu\ \approx \frac{e^{-\alpha k}}{\pi} \sum_{j=1}^{N} e^{-i\nu_j k} \Psi(\nu_j) \eta $$
where $\nu_j = \eta(j-1)$ and $\eta = B/N$.

1.1. Fast Fourier Transform (FFT)

While the latter expression for DFT is relatively easy to compute, there is a faster way that decreases computational costs: the **Fast Fourier Transform (FFT)**

The FFT maps a vector of the type $x = (x_j)_{j=1}^N$ to a vector $g(x_u)$ such that:

$$ g(x_u) = \sum_{j=1}^N e^{-i\frac{2\pi}{N}(j-1)(u-1)}x_j \ , \ for \ u=1,...,N. $$

Then, we can create a range $[-b,b]$ for the $Log K$ partitioned into N with λ equal spacing such that:

$$ k_u = -b + \lambda(u-1) \ , \ for \ u=1,...,N. $$

Substituting k_u and $\nu_j =\eta (j-1)$ in Eq. (1) yields

$$ C_T(k_u) \approx \frac{e^{-\alpha k_u}}{\pi} \sum_{j=1}^{N} e^{-i\nu_j k_u} \Psi(\nu_j) \eta = \frac{e^{-\alpha k_u}}{\pi} \sum_{j=1}^{N} e^{-i \lambda \eta (j-1)(u-1)} e^{ib\nu_j} \Psi(\nu_j) \eta $$

where usually $\lambda \eta = 2\pi / N$ and $e^{ib\nu_j} \Psi(\nu_j) \eta = x_j$ in FFT.

$C_T(k_u)$ gives the European call price for N strikes.

1.2. FFT with Simpson's rule

The trapezoidal rule is a valid and useful discretization technique. However, sometimes, the **Simpson rule** is preferred:

$$ \int_a^b f(x) \,dx \approx S(f,\eta) = \frac{\eta}{3}(f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + ... + 2f(x_{2N-2}) + 4f(x_{2N-1}) + f(x_{2N})) $$

Applying Simpson's rule in the same spirit as before on Eq. (1) yields

$$ C_T(k_u) = \frac{e^{-\alpha k_u}}{\pi} \sum_{j=1}^{N} e^{\frac{-i2\pi}{N}(j-1)(u-1)} e^{ib\nu_j} \Psi(\nu_j) \frac{\eta}{3}(3+(-1)^j - \delta_{j-1}) $$

where $\nu_j = \eta(j-1), \eta = B/N$ and $\delta_n = \mathbb{I}_{n=0}$.

*Note: This is the case for ITM options. For the OTM case, there are some stability issues that we will tackle in the future**

Now that we know about the basic functioning of DFT and FFT, let's implement it in code!

2. Pricing using FFT under Black-Scholes model

As a first example, let's use the FFT method on the Black-Scholes (BS) model. Obviously, this is something we will never do in practice, as for BS we do have a closed-form solution for the price of the option. However, this exercise will serve us well to illustrate the FFT process and in checking that numerical approximations using FFT converge to the actual closed-form price.

Let's start by importing some libraries:

In [ ]:
```
import numpy as np
from scipy import stats
from numpy.fft import fft   # This is, conveniently, a library that alrea
```

For this pricing exercise, let's think of a European call option with the following parameters:

In [ ]:
```
S0 = 100
K = 100
T = 1
r = 0.05
sigma = 0.2
```

2.1. Analytical Solution to BS Model

We will start with the already known analytical solution to the BS model:

In [ ]:
```
def BS_analytic_call(S0, K, T, r, sigma):
    """This function will provide the closed-form solution
    to the European Call Option price based on BS formula
    """

    d1 = (np.log(S0 / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T
    d2 = (np.log(S0 / K) + (r - 0.5 * sigma**2) * T) / (sigma * np.sqrt(T

    bs_call = S0 * stats.norm.cdf(d1, 0.0, 1.0) - K * np.exp(-r * T) * st
        d2, 0.0, 1.0
    )

    return bs_call
```

In [ ]:
```
print(
    " BS Analytical Call Option price will be $",
    BS_analytic_call(S0, K, T, r, sigma).round(4),
)
```

2.2. Black-Scholes Characteristic Function**

Fourier pricing methods require that we know the characteristic function of the process S_t (or some form of it like $s_t = log S_t$). In the case of a BS model (without dividends), the characteristic function is given by:

$$ \begin{equation*} \varphi^{BS} (u, T) = e^{((r-\frac{\sigma^2}{2})iu - \frac{\sigma^2}{2}u^2)T} \end{equation*} $$

Let's implement a function for it:

In [ ]:
```
def BS_characteristic_func(v, x0, T, r, sigma):
    """Computes general Black-Scholes model characteristic function
    to be used in Fourier pricing methods like Lewis (2001) and Carr-Mada
    """

    cf_value = np.exp(
        ((x0 / T + r - 0.5 * sigma**2) * 1j * v - 0.5 * sigma**2 * v**2)
    )

    return cf_value
```

2.3. Fast Fourier Transform (FFT) for Option Pricing

Next, we can implement Fast Fourier Transform (FFT) leveraging the 'fft' function included in the numpy library. Here you have the documentation for more details: https://numpy.org/doc/stable/reference/routines.fft.html

In [ ]:
```
def BS_call_FFT(S0, K, T, r, sigma):
    """European Call option price in BS under FFT"""

    k = np.log(K / S0)
    x0 = np.log(S0 / S0)
```

```
    g = 1  # Factor to increase accuracy
    N = g * 4096
    eps = (g * 150) ** -1
    eta = 2 * np.pi / (N * eps)
    b = 0.5 * N * eps - k
    u = np.arange(1, N + 1, 1)
    vo = eta * (u - 1)

    # Modifications to ensure integrability
    if S0 >= 0.95 * K:  # ITM Case
        alpha = 1.5
        v = vo - (alpha + 1) * 1j
        modcharFunc = np.exp(-r * T) * (
            BS_characteristic_func(v, x0, T, r, sigma)
            / (alpha**2 + alpha - vo**2 + 1j * (2 * alpha + 1) * vo)
        )

    else:
        alpha = 1.1
        v = (vo - 1j * alpha) - 1j
        modcharFunc1 = np.exp(-r * T) * (
            1 / (1 + 1j * (vo - 1j * alpha))
            - np.exp(r * T) / (1j * (vo - 1j * alpha))
            - BS_characteristic_func(v, x0, T, r, sigma)
            / ((vo - 1j * alpha) ** 2 - 1j * (vo - 1j * alpha))
        )

        v = (vo + 1j * alpha) - 1j

        modcharFunc2 = np.exp(-r * T) * (
            1 / (1 + 1j * (vo + 1j * alpha))
            - np.exp(r * T) / (1j * (vo + 1j * alpha))
            - BS_characteristic_func(v, x0, T, r, sigma)
            / ((vo + 1j * alpha) ** 2 - 1j * (vo + 1j * alpha))
        )

    # Numerical FFT Routine
    delt = np.zeros(N)
    delt[0] = 1
    j = np.arange(1, N + 1, 1)
    SimpsonW = (3 + (-1) ** j - delt) / 3
    if S0 >= 0.95 * K:
        FFTFunc = np.exp(1j * b * vo) * modcharFunc * eta * SimpsonW
        payoff = (fft(FFTFunc)).real
        CallValueM = np.exp(-alpha * k) / np.pi * payoff
    else:
        FFTFunc = (
            np.exp(1j * b * vo) * (modcharFunc1 - modcharFunc2) * 0.5 * e
        )
        payoff = (fft(FFTFunc)).real
        CallValueM = payoff / (np.sinh(alpha * k) * np.pi)

    pos = int((k + b) / eps)
    CallValue = CallValueM[pos] * S0

    return CallValue
```

The previous code contains a bunch of conditions and bounds (e.g., looking at close to ATM options). These are included to guarantee the 'good behavior' of the code. In

real life things may get messier, specially for deep OTM options where FFT algorithm can experience a harder time converging.

Importantly, the price we obtain under FFT converges to the analytical one obtained via the closed-form solution:

In [ ]:
```
print(
    " Fourier Call Option price via FFT is $", BS_call_FFT(S0, K, T, r, s
)
```

3. Execution Time

As you have seen, the two methods yield the same European option price. There are, however, differences in the execution time related to each.

In [ ]:
```
import datetime
```

In [ ]:
```
# BS Closed-form
begin = datetime.datetime.now()
price = BS_analytic_call(S0, K, T, r, sigma)
print(
    "BS closed-from price is $",
    price.round(4),
    ".Code took ",
    datetime.datetime.now() - begin,
)
```

In [ ]:
```
# BS via FFT
begin = datetime.datetime.now()
price = BS_call_FFT(S0, K, T, r, sigma)
print(
    "BS closed-from price is $",
    price.round(4),
    ".Code took ",
    datetime.datetime.now() - begin,
)
```

4. Conclusion

Well done! Now you know how to implement the FFT method for option pricing in the Black-Scholes framework. In the next lesson, we will show you how to do this in continuous time, under the Lewis (2001) approach.

References

- Hilpisch, Yves. *Derivatives Analytics with Python: Data Analysis, Models, Simulation, Calibration and Hedging*. John Wiley & Sons, 2015.
- Carr, Peter, and Dilip Madan. "Option valuation using the fast Fourier transform." Journal of computational finance 2.4 (1999): 61-73.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.